AgPitch

Live football match insights that explain *why* the game is changing

Project Report

Roshan Palem · October 2026

Hackathon	Inside the Game: Developer Hackathon (Microsoft + Premier League)
Challenge	The Synthetic Match Insights Engine for Premier League Studio
Team	Solo
Live app	`https://thankful-rock-0c1b05c0f.6.azurestaticapps.net`
Demo video	`https://youtu.be/Nov0eNelsuE`
Code	`https://github.com/roshanroy7/AgPitch`

1. The problem

When I watch a match I get the score and a few stats, but nobody tells me *why* the game just turned. Why did a team suddenly start losing the ball? Was that save actually important? And a stats nerd and a casual fan want completely different answers to the same question.

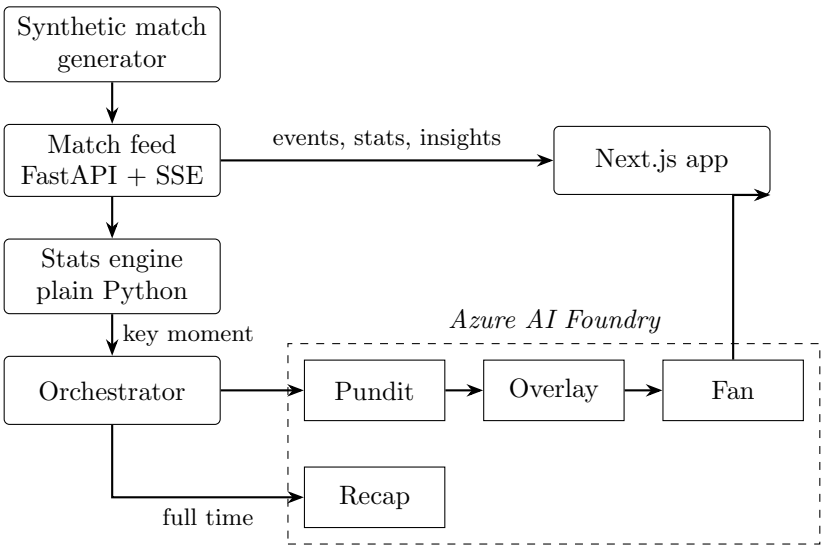
The challenge asked for an engine that takes a stream of match events and turns it into live insights a broadcaster could use. My goal was simple: explain the match as it happens, and explain it differently for each viewer.

2. What I built

AgPitch replays a full 90-minute synthetic match as if it were live, flags the moments that matter, and has four AI agents on Azure AI Foundry explain each one, turn it into an on-screen graphic, and rewrite it for the person watching.

- Live score, clock, scorers and a commentary feed (shots, saves, blocks, corners, counter-attacks, fouls, cards, injuries)
- AI insight cards that appear at the exact minute of each key moment, with a "Why it matters" explanation
- Three viewing modes: casual fan, analyst and player focus
- Favourite club and favourite player
- Three languages: English, Spanish and Portuguese (every line of the interface, not just the AI text)
- Stats tab with top stats and a clickable shot map
- Full-time recap written for that viewer
- Changing a setting mid-match continues the stream from the current minute

3. Architecture



Stage	Component	What it does
Ingest	`main.py`	Replays the match event by event and streams it with Server-Sent Events
Interpret	`stats.py`	Computes all numbers and flags key moments using rules, no AI
Explain	Pundit agent	Explains why the moment matters: control vs chaos, pressure, momentum, rhythm
Render	Overlay agent	Produces a timed, machine-readable graphic (type, headline, stat, duration, priority)
Personalise	Fan + Recap agents	Rewrites the insight for the viewer's mode, club, player and language; writes the full-time recap

4. Synthetic data

No real Premier League data is used. `generator.py` builds a full match (Kryptonites vs Ultras) from a scripted storyline: Kryptonites control the first half and score, Ultras switch to a high press after the break, equalise and win it late (2–1).

Around that storyline it generates possession chains, passes with pitch coordinates, shots (saved, blocked, off target), corners, clearances, counter-attacks, fouls, cards and injuries. A fixed random seed means it is the same match every run, which made testing and the demo repeatable. All teams and players are made up.

Events	1,396	Counter-attacks	6
Goals	3	Yellow cards	3
Saves	15	Injuries	2
Corners	10	Key moments flagged	about 46

5. The stats engine

Everything numeric is calculated in plain Python, not by the AI: possession (overall and last 10 minutes), pass accuracy, pass distance, a 0 to 10 pass difficulty rating, passes under pressure, shot speed, saves, corners, counter-attacks and cards.

Key moments are flagged by rules, each with a cooldown so the same thing is not flagged twice in a row:

- **Turnover streak:** 8 turnovers in 5 minutes
- **Pressure spike** and **momentum swing** (an 18-point shift)
- **Speed alert** (33 km/h and above) and **rocket shot** (100 km/h and above)
- **Big save:** a shot of 90 km/h or more saved
- **Counter-attack**, **cards**, **goals**, **quality pass**, **50-pass milestone**, half time and full time

6. Agent design

All four agents run on **Azure AI Foundry Agent Service** with **gpt-5-mini**. They are created from code (`python -m backend.app.agents setup`), so their instructions are versioned in the repository.

Agent	Input	Output
Pundit	Key moment + live stats	Theme, importance (1 to 5), insight
Overlay	Moment + Pundit analysis	Graphic type, headline, stat, duration, priority
Fan	Overlay + analysis + viewer profile	Headline, body, stat, why it matters, tone
Recap	Final stats + all moments + profile	Title, summary, turning point, player of the match

Shared rules for every agent: the data is synthetic, never mention real clubs or players, only use numbers from the input, and always reply in JSON.

7. Key design decisions

1. **The AI never does the maths.** LLMs are bad at counting, so every number comes from the stats engine and the agents only explain it.
2. **Only key moments go to the agents.** About 46 calls per match instead of about 1,400. Faster and much cheaper.
3. **Shared vs personal work.** Pundit and Overlay run once per moment and are shared by every viewer. Fan runs once per moment per viewer profile, so ten viewers with the same profile cost one call.
4. **The stream never waits for the AI.** Each moment becomes a background task. A card appears instantly in an "Analysing" state and fills in when the agents answer.
5. **Caching everywhere.** Results are cached in memory and on disk, keyed by a fingerprint of the match file, so a regenerated match never reuses old insights.
6. **Warm-up before a broadcast.** A `/warmup` endpoint runs the whole match through the agents in parallel (8 calls at a time), so every insight is ready exactly on time.
7. **Fail safe.** If Azure fails, the viewer still gets a basic card from the stats engine, and the stream never breaks.
8. **JSON in, JSON out.** The frontend never has to guess what an agent returned.

8. Deployment

Part	Azure service	Notes
AI agents	Azure AI Foundry Agent Service	gpt-5-mini deployment
Backend	Azure Container Apps	FastAPI + SSE; Docker image built by GitHub Actions
Frontend	Azure Static Web Apps	Next.js static export
Auth	Managed Identity	No API keys in the code; `az login` locally

CORS is restricted to localhost and the deployed frontend.

9. Challenges and how I solved them

- **Insights arrived about 20 seconds late.** An agent chain takes 10 to 20 seconds, so cards appeared after the moment had passed. I added "Analysing" cards that appear at the right minute, sorted cards by match time instead of arrival time, and added the warm-up cache.
- **The free trial blocked the normal container build.** `az containerapp up` failed because ACR Tasks are not allowed on the free trial. I switched to building the Docker image with GitHub Actions, pushed it to GitHub Container Registry, and deployed that image to Container Apps.
- **Language quality.** I tried Telugu (my own language) first, but the output was not good enough, so I dropped it rather than ship something I would not watch myself. Spanish and Portuguese worked well. "Why it matters" stayed in English at first, so I added a `why` field to the Fan agent's output, written in the viewer's language.
- **Design.** My first interface did not feel like a football app. I redesigned it around tabs (Live, Stats, Recap), a match header, a settings pop-up and the two clubs' colours.

10. Limitations

- One synthetic match only; there is no real event feed yet.
- Insights for a new viewer profile take 10 to 20 seconds unless that profile was warmed up first.
- The translations have not been checked by native speakers.
- It runs as a single backend instance, sized for a demo, not for thousands of viewers.

11. What I would do next

- Connect a real event feed, and eventually generate events from video
- A momentum chart across the whole match
- More languages, each checked by native speakers
- Scale the backend out with a shared cache (for example Redis) so many instances can serve viewers
- Evaluate the agents' output with Azure AI Foundry evaluations

12. What I learned

- How to design a multi-agent system where each agent has one clear job and a strict JSON contract

- Why deterministic code should own the facts, and AI should own the explanation
- Streaming with Server-Sent Events, background tasks and caching to hide AI latency
- Deploying a full stack on Azure (Container Apps, Static Web Apps, Managed Identity), including working around free-trial limits

Tech stack

Python 3.13, FastAPI, Server-Sent Events, `azure-ai-projects`, `azure-identity`, Azure AI Foundry Agent Service (gpt-5-mini), Next.js 16, React 19, plain CSS, Docker, GitHub Actions, Azure Container Apps, Azure Static Web Apps.